JOTA manuscript No.
(will be inserted by the editor)

Ky Fan Norms and Beyond: Dual Norms and Combinations for Matrix Optimization

Alexey Kravatskiy · Ivan Kozyrev · Nikolai Kozlov · Alexander Vinogradov · Daniil Merkulov · Ivan Oseledets

Received: date / Accepted: date

Abstract In this article, we explore the use of various matrix norms for optimizing functions of weight matrices, a crucial problem in deep learning. Moving beyond the spectral norm that underlies the Muon update, we leverage the duals of the Ky Fan norms to introduce the *Fanion* family of linear minimization oracle (LMO) algorithms, which are closely related to Muon, ν-SAM, and Dion. Staying inside the LMO, we construct the families of *F-Fanions* and *S-Fanions*, whose updates are convex combinations of the updates of Fanions and Normalized SGD or SignSGD, respectively. The most promising algorithms in

Alexey Kravatskiy, Corresponding author
MIRIAI
Russia
E-mail: kravatskii.a@miriai.org

Ivan Kozyrev
MIPT, INM RAS
Russia
E-mail: kozyrev.in@phystech.edu

Nikolai Kozlov
MIPT
Russia
E-mail: kozlov.na@phystech.edu

Alexander Vinogradov
MIPT
Russia
E-mail: vinogradov.am@phystech.edu

Daniil Merkulov
MIPT, Skoltech, HSE, AI4Science
Russia
E-mail: daniil.merkulov@phystech.edu

Ivan Oseledets
AIRI, Skoltech, INM RAS
Russia
E-mail: i.oseledets@skoltech.ru

these families are *F-Muon* and *S-Muon*. By conducting an extensive empirical study of all three algorithm families across a wide range of tasks and settings, we demonstrate that F-Muon and S-Muon consistently match Muon's performance, while outperforming Muon on a synthetic smooth convex problem.

Keywords Matrix optimization · Linear minimization oracle · Muon optimizer · Ky Fan norms

Mathematics Subject Classification (2000) 90C06 · 68T07 · 15A60 · 90C30

1 Introduction

Minimizing loss functions in unprecedentedly high-dimensional spaces has recently become an integral and crucial part of training large language models. Hence, new scalable, time- and memory-efficient algorithms have been demanded. Besides the well-known Adam [22] and AdamW [28], recently proposed Muon [19] has shown promising results on training very large models [27]. Its key difference from Adam and AdamW is that it has been constructed specifically for optimizing functions of weight matrices, which are common in deep learning.

From a theoretical perspective, a key innovation of Muon was its principled derivation of the update rule, which emerged as the solution to an optimization problem constrained by the RMS-to-RMS norm (a scaled version of the spectral norm) [3].

Motivated by the success of Muon, many generalizations and variations of it were proposed. Among the notable ones are Scion [32], Dion [1] and Gluon [36]. Those works try to explain Muon's efficiency and establish convergence bounds. One central question, however, remains unanswered:

Can the employment of a norm other than an operator norm in Muon-like algorithms yield comparable or superior empirical results?

In this article, we tackle this question by demonstrating that there are many viable non-operator norms. We leverage the family of norms dual to Ky Fan k-norms to derive a new family of *Fanions*, algorithms with low-rank updates. This approach theoretically explains the backbone of Dion's update [1] and generalizes the memory-motivated application of the nuclear norm to Sharpness-Aware Minimization [31]. As was done with Muon, we develop an effective procedure for computing Fanions' updates: the Lanczos algorithm is the solution (see Sect. 5).

Working with dual norms and various convex combinations of norms, we construct the families of *F-Fanions* and *S-Fanions*, which are hybrids of Fanions with Normalized SGD and SignSGD, respectively.

In Sect. 6, we compare the performance of these algorithm families on various model and real-world problems:

- Synthetic smooth convex problem with a matrix argument
- CIFAR-10 airbench [21]

– Pre-training NanoGPT and GPT-2 Medium on the FineWeb dataset [18]
– Fine-tuning NanoGPT on TinyStories dataset [10]

Our experiments reveal important insights into the role of matrix norms in optimization. First, using the example of Neon (the rank-one Fanion), we show that not every LMO-based algorithm is effective, despite sharing the same asymptotics in the general bounds of [23] and [36]. This suggests that existing theoretical guarantees should be refined to better explain empirical performance.

Second, our experiments on real-world tasks demonstrate that the choice of underlying matrix norm is remarkably flexible. On CIFAR-10 airbench, properly-tuned F-Muon and S-Muon achieve $94.02 \pm 0.13\%$ and $94.03 \pm 0.13\%$ accuracy, matching Muon's $94.01 \pm 0.13\%$ performance. After 1750 iterations on NanoGPT training, F-Muon achieves 3.281 cross-entropy loss, while Muon achieves 3.279. Similarly, after 5960 iterations of GPT-2 Medium training, we get 2.9215 with F-Muon and 2.9198 with Muon. Finally, S-Muon matches Muon in fine-tuning NanoGPT on TinyStories, while F-Muon is far more robust to learning rate choice than Muon. These results show that Muon-like algorithms can maintain competitive performance even when the underlying norm constraint is significantly modified, providing an affirmative answer to the central question posed above. Moreover, the tools from Sect. 4 give researchers considerable flexibility in designing algorithms that need not be direct modifications of Muon.

2 Preliminaries: Linear Minimization Oracle Framework

Training a neural network is essentially an optimization of a function of several weight matrices and a few vectors. Let us start by considering the problem of minimizing a differentiable function of a *single* matrix (the connection to the general case is presented in Sect. A):

$$F(\cdot)\colon \mathbb{R}^{m\times n} \to \mathbb{R}\,, \qquad F(\boldsymbol{X}) \to \min_{\boldsymbol{X}\in\mathbb{R}^{m\times n}} \tag{1}$$

We equip the matrix space $\mathbb{R}^{m\times n}$ with a standard dot product $\langle\cdot,\cdot\rangle \to \mathbb{R}$ and a norm $\|\cdot\|\colon \mathbb{R}^{m\times n} \to \mathbb{R}_+$, which does not have to coincide with the Frobenius norm $\|\cdot\|_{\mathrm{F}} = \sqrt{\langle\cdot,\cdot\rangle}$. The dual norm $\|\cdot\|^\dagger\colon \mathbb{R}^{m\times n} \to \mathbb{R}_+$ that is associated with $\|\cdot\|$ is defined as

$$\|\boldsymbol{M}\|^\dagger = \sup_{\boldsymbol{D}\in\mathbb{R}^{m\times n}\colon \|\boldsymbol{D}\|\le 1} \langle \boldsymbol{M}, \boldsymbol{D}\rangle\,. \tag{2}$$

Such problems can be solved with an iterative algorithm based on the Linear Minimization Oracle (LMO):

$$\mathrm{LMO}(\boldsymbol{M}^t) \in \arg\min_{\boldsymbol{D}\in\mathcal{S}} \langle \boldsymbol{M}^t, \boldsymbol{D}\rangle\,, \tag{3}$$

where $\boldsymbol{M}^t$ is the effective update direction and $\mathcal{S} \subset \mathbb{R}^{m \times n}$ is a constraint set. The algorithm proceeds as follows:

$$\begin{aligned} \boldsymbol{B}^t &= \beta \boldsymbol{B}^{t-1} + (1-\beta)\nabla F(\boldsymbol{X}^t) \quad \text{(momentum buffer)}, \\ \boldsymbol{M}^t &= \begin{cases} \nabla F(\boldsymbol{X}^t) & \text{(no momentum)}, \\ \boldsymbol{B}^t & \text{(heavy ball)}, \\ \nabla F(\boldsymbol{X}^t) + \beta \boldsymbol{B}^t & \text{(approximate Nesterov)}, \end{cases} \\ \boldsymbol{X}^{t+1} &= \boldsymbol{X}^t + \gamma_t \mathrm{LMO}\left(\boldsymbol{M}^t\right), \end{aligned} \tag{4}$$

where $\beta \in [0, 1)$ is the momentum coefficient and $\nabla F(\boldsymbol{X}^t)$ is either full or stochastic gradient. Throughout our experiments, we employ approximate Nesterov momentum, which matches the implementation in Muon and PyTorch SGD (`nesterov=True`). It uses the current gradient rather than the true lookahead gradient, trading theoretical guarantees for computational efficiency. From here on, we refer to the stochastic gradient as just the gradient if not otherwise specified.

We are particularly interested in the case when $\mathcal{S}$ is a unit ball in some norm $\|\cdot\|$:

$$\mathcal{S} = \mathcal{B}_{\|\cdot\|} = \{\boldsymbol{D} \in \mathbb{R}^{m \times n} : \|\boldsymbol{D}\| \leq 1\}.$$

In this case,

$$\arg\min_{\boldsymbol{D} \in \mathcal{S}} \langle \boldsymbol{M}^t, \boldsymbol{D} \rangle = -\{\boldsymbol{D} \in \mathcal{B}_{\|\cdot\|} : \langle \boldsymbol{M}^t, \boldsymbol{D} \rangle = \|\boldsymbol{M}^t\|^\dagger\},$$

and the update for $\boldsymbol{X}^{t+1}$ in (4) simplifies to

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \{\boldsymbol{D} \in \mathcal{B}_{\|\cdot\|} : \langle \boldsymbol{M}^t, \boldsymbol{D} \rangle = \|\boldsymbol{M}^t\|^\dagger\}. \tag{5}$$

Using this formula, it is easy to compute updates for algorithms induced by various norms $\|\cdot\|$ [4, 32]:

Frobenius norm and Normalized SGD When the norm $\|\cdot\|$ is the Frobenius norm $\|\cdot\|_{\mathrm{F}}$, (5) turns into

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \frac{\boldsymbol{M}^t}{\|\boldsymbol{M}^t\|_{\mathrm{F}}}, \tag{6}$$

which recovers Normalized SGD (NSGD).

Spectral norm and Muon When the norm is the spectral norm $\|\cdot\|_2$, we get

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \boldsymbol{U}\boldsymbol{V}^\top, \tag{7}$$

which is Muon without the $\sqrt{m/n}$ factor. Here, $\boldsymbol{M}^t = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$ is the Singular Value Decomposition (SVD) of $\boldsymbol{M}^t$ ($\boldsymbol{U} = [\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_r]$, $\boldsymbol{\Sigma} = \mathrm{diag}(\sigma_1, \sigma_2, \ldots, \sigma_r)$, and $\boldsymbol{V} = [\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_r]$). Muon can be recovered by taking the RMS-to-RMS norm: $\sqrt{n/m}\|\cdot\|_2$.

Chebyshev norm and SignSGD When the norm is the Chebyshev norm $\|\cdot\|_{\mathrm{C}}$ (also known as the infinity norm, $\|A\|_{\mathrm{C}} \equiv \max_{i,j}|A_{ij}|$), we get

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \operatorname{sign}(\boldsymbol{M}^t)\,, \tag{8}$$

which recovers SignSGD [5]. Here, $\operatorname{sign}(\boldsymbol{M}^t)$ denotes the element-wise sign function. SignSGD is particularly notable for its communication efficiency in distributed training, as it compresses gradients to 1 bit per parameter.

3 Beyond the Spectral Norm: Fanions

3.1 The Rank Gap Problem

Having examined the Frobenius norm (NSGD), spectral norm (Muon), and Chebyshev norm (SignSGD), all of which produce full-rank updates, we turn to the nuclear norm $\|\cdot\|_*$.

Lemma 3.1 *When $\|\cdot\| = \|\cdot\|_*$, (5) becomes*

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \boldsymbol{u}_1 \boldsymbol{v}_1^\top\,. \tag{9}$$

Proof Since $\|\cdot\|_*^\dagger = \|\cdot\|_2$, the goal is to reach $\langle \boldsymbol{M}^t, \boldsymbol{D}\rangle = \sigma_1$ in (5). Note that $\boldsymbol{D} = \boldsymbol{u}_1\boldsymbol{v}_1^\top$ delivers this value. Indeed, $\|\boldsymbol{D}\|_* = 1$ and by the trace property and orthogonality of the singular vectors,

$$\langle \boldsymbol{M}^t, \boldsymbol{D}\rangle = \langle \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top, \boldsymbol{u}_1\boldsymbol{v}_1^\top\rangle = \operatorname{tr}\operatorname{diag}(\sigma_1, 0, \ldots, 0) = \|\boldsymbol{M}^t\|_2\,,$$

which completes the proof. □

We call this algorithm *Neon*. The nuclear norm yields rank-one updates, in stark contrast to the full-rank updates of Muon, NSGD, and SignSGD. This raises a natural question: can we derive algorithms with updates of intermediate ranks?

Schatten norms Cesista [7] considered Schatten-p norms:

$$\|\boldsymbol{M}^t\|_{S_p} = \left(\sum_{i=1}^{\min(m,n)} \sigma_i^p\right)^{1/p}\,,$$

which produce the updates

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \boldsymbol{U} \frac{\operatorname{diag}\left(\sigma_1^{q-1}, \ldots, \sigma_{\min(m,n)}^{q-1}\right)}{\left(\sum_{i=1}^{\min(m,n)} \sigma_i^q\right)^{\frac{q-1}{q}}} \boldsymbol{V}^\top$$

where p and q satisfy $p^{-1} + q^{-1} = 1$. This formula recovers Neon when $p \to 1$ (provided that $\sigma_1 > \sigma_2$, which holds with probability 1 on real data), NSGD when $p = 2$, and Muon when $p \to \infty$.

However, Schatten norms do not fill the rank gap: when $p > 1$, the update has full rank, while when $p = 1$, its rank equals 1. Moreover, computing the update for $p \neq 1, 2, \infty$ appears to require knowing all σ_i, making the problem as hard as computing the full SVD.

Ky Fan norms Another family of matrix norms could offer a solution: Ky Fan norms. For $k \in \{1, \ldots, \min(m, n)\}$, the Ky Fan k-norm $\|\cdot\|_{\text{KF-}k}$ equals $\sum_{i=1}^{k} \sigma_i$, the sum of the k largest singular values. Notable special cases include the Ky Fan 1-norm (the spectral norm) and the Ky Fan $\min\{m, n\}$-norm (the nuclear norm).

To derive the update formula for arbitrary k, we use the expression for the norm dual to the Ky Fan k-norm (see, for instance, [6], p. 96):

$$\|\cdot\|^{\dagger}_{\text{KF-}k} = \max\left\{\frac{1}{k}\|\cdot\|_*, \|\cdot\|_2\right\}.$$

According to (5), we need to find $\boldsymbol{D}$ such that $\|\boldsymbol{D}\|_{\text{KF-}k} = 1$ and

$$\langle \boldsymbol{M}^t, \boldsymbol{D}\rangle = \max\left\{\frac{1}{k}\sum_{i=1}^{\min(m,n)} \sigma_i,\ \sigma_1\right\}.$$

Neon's update $\boldsymbol{D} = \boldsymbol{u}_1\boldsymbol{v}_1^\top$ achieves σ_1, while Muon's scaled update $\boldsymbol{D} = \frac{1}{k}\boldsymbol{U}\boldsymbol{V}^\top$ achieves $\frac{1}{k}\sum_{i=1}^{\min(m,n)} \sigma_i$. Thus, the update is either

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \boldsymbol{u}_1\boldsymbol{v}_1^\top \text{ or } \boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \frac{\gamma_t}{k}\boldsymbol{U}\boldsymbol{V}^\top,$$

depending on which of those two expressions achieves a smaller residual on the linear function $L(\boldsymbol{X}^{t+1}) := F(\boldsymbol{X}^t) + \langle \boldsymbol{M}^t, \boldsymbol{X}^{t+1} - \boldsymbol{X}^t\rangle$. The Ky Fan norms thus fail to close the rank gap: the resulting update is either rank-one or full-rank.

3.2 Solution: Duals to Ky Fan Norms

Unlike Schatten norms, which satisfy $\|\boldsymbol{M}^t\|^{\dagger}_{S_p} = \|\boldsymbol{M}^t\|_{S_q}$ (for $p^{-1} + q^{-1} = 1$) and are thus closed under dualization, Ky Fan norms are generally not. Only two exceptional cases exist: the dual to the Ky Fan 1-norm $\|\cdot\|_{\text{KF-1}}$ (the spectral norm) is the Ky Fan $\min(m, n)$-norm $\|\cdot\|_{\text{KF-}\min(m,n)}$ (the nuclear norm), and vice versa. Thus, working with duals of Ky Fan norms can open up new possibilities:

Lemma 3.2 *When $\|\cdot\| = \|\boldsymbol{M}^t\|^{\dagger}_{KF\text{-}k}$, (5) turns into:*

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \sum_{i=1}^{k} \boldsymbol{u}_i\boldsymbol{v}_i^\top. \tag{10}$$

Proof Since $\|\cdot\|^\dagger = \|\cdot\|^{\dagger\dagger}_{\text{KF-}k} = \|\cdot\|_{\text{KF-}k}$, the goal is to reach $\langle \boldsymbol{M}^t, \boldsymbol{D}\rangle = \|\boldsymbol{M}^t\|_{\text{KF-}k}$ in (5). Note that $\boldsymbol{D} = \sum_{i=1}^k \boldsymbol{u}_i \boldsymbol{v}_i^\top$ attains this value. Indeed,

$$\langle \boldsymbol{M}^t, \boldsymbol{D}\rangle = \langle \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top, \sum_{i=1}^k \boldsymbol{u}_i \boldsymbol{v}_i^\top\rangle = \sum_{i,j=1}^{r,k} \langle \boldsymbol{u}_i \sigma_i \boldsymbol{v}_i^\top, \boldsymbol{u}_j \boldsymbol{v}_j^\top\rangle = \sum_{i=1}^k \sigma_i = \|\boldsymbol{M}^t\|_{\text{KF-}k}\,,$$

which completes the proof. □

These updates define the *Fanion* family of LMO-based algorithms, each operating under a norm $\|\boldsymbol{M}^t\|^\dagger_{\text{KF-}k}$. We denote the algorithm for a particular k as *Fanion-k*. This family elegantly bridges the rank gap, providing updates of any intermediate rank k. For a visualization of the ball in the $\|\cdot\|^\dagger_{\text{KF-}k}$ norm, see Sect. D.2.

Connection to existing algorithms The Fanion family unifies several known algorithms:

- **Neon** is Fanion-1 (rank-one updates)
- **Muon** is Fanion-min$\{m, n\}$ (full-rank updates)
- **Dion** (unsharded): The rank-r Dion (Algorithm 1 from [1]) without error feedback and without scaling of the update is actually Fanion-r (see [29], where Dion is written in a notation more similar to ours)

In Sect. 5, we discuss how to efficiently compute Fanion updates using the Lanczos algorithm.

4 Conic Combination of LMO-Algorithms Is an LMO-Algorithm

The approaches to designing new matrix LMO-algorithms are not limited to applying norms dual to Ky Fan k-norms. We now consider linear combinations of LMO-algorithms and show that these combinations are themselves LMO-algorithms.

4.1 General Case

We begin with a known property of a conic combination of norms (see, e.g., [44, Table 1]).

Lemma 4.1 *Let $\|\cdot\|_{(1)}, \ldots, \|\cdot\|_{(n)}$ be norms on a finite-dimensional Euclidean space, and let $\alpha_1, \ldots, \alpha_n$ be non-negative reals. Define*

$$\|\cdot\| := \sum_{i=1}^n \alpha_i \|\cdot\|_{(i)}\,.$$

Then the dual unit ball of $\|\cdot\|$ *satisfies*

$$\mathcal{B}_{\|\cdot\|^\dagger} = \sum_{i=1}^{n} \alpha_i \mathcal{B}_{\|\cdot\|_{(i)}^\dagger} ,$$

where $\sum$ *denotes the Minkowski sum and* $\mathcal{B}_{\|\cdot\|_{(i)}^\dagger}$ *is the unit ball of the dual norm* $\|\cdot\|_{(i)}^\dagger$.

A proof is provided in the appendix (see Sect. B).

Lemma 4.2 *Let* $\|\cdot\|_{(1)}, \ldots, \|\cdot\|_{(n)}$ *be norms on a finite-dimensional Euclidean space, and let* $\alpha_1, \ldots, \alpha_n$ *be non-negative reals. Consider Linear Minimization Oracles* $\mathrm{LMO}_{\|\cdot\|_{(1)}}, \ldots, \mathrm{LMO}_{\|\cdot\|_{(n)}}$, *which correspond to the unit balls of these norms. Then,* $\sum_{i=1}^{n} \alpha_i \mathrm{LMO}_{\|\cdot\|_{(i)}}$ *is the LMO corresponding to the norm* $\|\cdot\|$ *dual to the norm given by* $\sum_{i=1}^{n} \alpha_i \|\cdot\|_{(i)}^\dagger$.

Proof Using Lemma 4.1 and the biduality property $\|\cdot\|^{\dagger\dagger} = \|\cdot\|$, we obtain the unit ball representation: $\mathcal{B}_{\|\cdot\|} = \sum_{i=1}^{n} \alpha_i \mathcal{B}_{\|\cdot\|_{(i)}}$. The linear minimization problem over this ball can thus be transformed as follows:

$$\arg\min_{\boldsymbol{D} \in \mathcal{B}_{\|\cdot\|}} \langle \boldsymbol{M}, \boldsymbol{D} \rangle = \arg\min_{\boldsymbol{D}_1 \in \alpha_1 \mathcal{B}_{\|\cdot\|_{(1)}}, \ldots, \boldsymbol{D}_n \in \alpha_n \mathcal{B}_{\|\cdot\|_{(n)}}} \langle \boldsymbol{M}, \sum_{i=1}^{n} \boldsymbol{D}_i \rangle = \sum_{i=1}^{n} \arg\min_{D_i \in \mathcal{B}_{\|\cdot\|_{(i)}}} \langle \boldsymbol{M}, \boldsymbol{D}_i \rangle ,$$

where the last summation denotes the Minkowski sum. This immediately implies

$$\sum_{i=1}^{n} \alpha_i \mathrm{LMO}_i \in \arg\min_{\boldsymbol{D} \in \mathcal{B}_{\|\cdot\|}} \langle \boldsymbol{M}, \boldsymbol{D} \rangle ,$$

completing the proof. □

Applying this result to optimization algorithms yields the following corollary.

Corollary 4.1 *Let there be a finite family of LMO-based algorithms indexed by* $i = 1, \ldots, n$, *where the* $\boldsymbol{X}^{t+1}$ *update of the* i*-th algorithm is defined by*

$$\boldsymbol{X}^{t+1} - \boldsymbol{X}^t = \gamma_t \mathrm{LMO}_{\|\cdot\|_{(i)}}(\boldsymbol{M}^t) ,$$

and $\mathrm{LMO}_{\|\cdot\|_{(i)}}$ *corresponds to the unit ball of norm* $\|\cdot\|_{(i)}$. *Then, for arbitrary non-negative* $\alpha_1, \ldots, \alpha_n$, *the algorithm with the update given by*

$$\boldsymbol{X}^{t+1} - \boldsymbol{X}^t = \gamma_t \sum_{i=1}^{n} \alpha_i \mathrm{LMO}_{\|\cdot\|_{(i)}}(\boldsymbol{M}^t)$$

is an LMO-algorithm itself, with LMO corresponding to the unit ball of the norm $\|\cdot\|$ *dual to the norm given by* $\sum_{i=1}^{n} \alpha_i \|\cdot\|_{(i)}^\dagger$.

4.2 Frobeniusize Them: F-Muon and F-Neon

We now construct concrete examples of algorithms obtained via linear combinations of LMO-algorithms. By Corollary 4.1, these combinations are themselves LMO-algorithms.

Combining Fanions with NSGD yields a family of algorithms with updates

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \left(\alpha \sum_{i=1}^{k} \boldsymbol{u}_i \boldsymbol{v}_i^\top + (1-\alpha) \frac{\boldsymbol{M}^t}{\|\boldsymbol{M}^t\|_{\mathrm{F}}} \right) . \tag{11}$$

Recall that Fanion-k operates under the dual to the Ky Fan k-norm, while NSGD operates under the self-dual Frobenius norm. By Corollary 4.1, this combination defines an LMO-algorithm with norm $\|\cdot\|^\dagger_{\mathrm{F-KF-k}}$, where

$$\|\cdot\|_{\mathrm{F-KF-k}} = \alpha \|\cdot\|_{\text{KF-}k} + (1-\alpha)\|\cdot\|_{\mathrm{F}} . \tag{12}$$

We call this family *F-Fanions*.

The extreme members of this family are *F-Neon* (with $k = 1$) and *F-Muon* (with $k = \min\{m, n\}$). Additional information and visualizations for the F-Muon generating $\|\cdot\|^\dagger_{\mathrm{F}*} = \|\cdot\|^\dagger_{\mathrm{F-KF-min\{m,n\}}}$ and the F-Neon generating $\|\cdot\|^\dagger_{\mathrm{F2}} = \|\cdot\|^\dagger_{\mathrm{F-KF-1}}$ norms appear in the appendix (see (16), (17), Fig. 8).

4.3 Add SignSGD: S-Muon and S-Neon

Similarly, combining Fanion-k with SignSGD yields *S-Fanion-k* with the update

$$\boldsymbol{X}^{t+1} = \boldsymbol{X}^t - \gamma_t \left(\alpha \sum_{i=1}^{k} \boldsymbol{u}_i \boldsymbol{v}_i^\top + (1-\alpha)\eta \operatorname{sign}(\boldsymbol{M}^t) \right) , \tag{13}$$

where `sign_lr_coeff` η is a scaling coefficient for SignSGD.

By Corollary 4.1, this defines an LMO-algorithm with norm $\|\cdot\|^\dagger_{\mathrm{C}^\dagger\mathrm{-KF-k}}$, where

$$\|\cdot\|_{\mathrm{C}^\dagger\mathrm{-KF-k}} = \alpha \|\cdot\|_{\text{KF-}k} + \frac{1-\alpha}{\eta} \|\cdot\|^\dagger_{\mathrm{C}} . \tag{14}$$

The extreme members of this family are *S-Neon* (with $k = 1$) and *S-Muon* (with $k = \min\{m, n\}$).

5 Computing the Updates

We employ the thick-restart Lanczos method for the symmetric eigenvalue problem (TRLan) to compute the low-rank updates of Fanions. We apply TRLan to either $\boldsymbol{M}^{t\top}\boldsymbol{M}^t$ or $\boldsymbol{M}^t\boldsymbol{M}^{t\top}$, selecting whichever matrix is smaller. We use the CuPy implementation of `cupy.sparse.linalg.svds` [35], which internally relies on TRLan [43].

TRLan is specifically designed for efficiently approximating the largest singular values and corresponding singular vectors of very large matrices. The thick-restart strategy retains the most informative Ritz vectors across restarts, which dramatically accelerates convergence while maintaining moderate memory consumption. TRLan is particularly well-suited to our GPU setting because its dominant computational cost consists of a modest number of highly parallelizable matrix-vector multiplications (matvecs), and it avoids full reorthogonalization against the entire Krylov basis by employing short recurrence relations combined with thick restarting.

The per-cycle complexity is $\mathcal{O}(mn^2 + n^2d + nd^2)$, where $m \geq n$ are the dimensions of the target matrix and d is the size of the retained subspace ($d \ll n$ typically).

In Table 1, we compare TRLan against randomized SVD (RSVD) and simple power iterations for computing the rank-k update used in Fanion-k and related algorithms. Experiments are performed on dense random matrices with i.i.d. $\mathcal{N}(0, 1)$ entries using CPU implementations for fair comparison. We report:

- err_1: relative error in the Frobenius norm of $\sum_i^k \boldsymbol{u}_i \boldsymbol{v}_i^T$,
- err_2: relative error in the Frobenius norm of $\sum_i^k \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$.

On 500×500 matrices, TRLan and RSVD require comparable wall-clock time, but TRLan delivers orders-of-magnitude lower error with far fewer matvecs. On larger 5000×5000 matrices, this advantage becomes even more pronounced: TRLan is 3-4 times faster than RSVD while using $\sim$30 times fewer matvecs at comparable or superior accuracy.

An interesting empirical observation is that RSVD tends to approximate the *singular values* themselves reasonably well, but the reconstructed low-rank matrix exhibits noticeable deviation from the truncated SVD. In contrast, TRLan provides an excellent approximation to the truncated SVD matrix itself (low err_2), albeit at the cost of occasionally less accurate individual singular values. This makes TRLan the preferred choice for algorithms like Neon/Fanion-k that only require the low-rank term $\sum \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^\top$, but less suitable for methods (e.g., Dion) that explicitly require accurate σ_i for error feedback or step-size control.

A current practical limitation is the absence of a native PyTorch implementation of thick-restart Lanczos; existing PyTorch-based randomized SVD routines cannot match TRLan's accuracy-efficiency combination for the matrix reconstruction task.

For reference, Table 2 presents results for the Newton-Schulz polar decomposition iteration on the same matrices, where err_1 is the relative error of $\boldsymbol{U}\boldsymbol{V}^T$ (29-30 iterations to converge, resulting in a significantly higher matvec count than TRLan).

Table 1 Comparison of methods for computing rank-k updates on dense random matrices (CPU, double precision). Lower is better in all columns.

Matrix sizes	k	Method	Time (s)	Matvecs	Iterations	err_1	err_2
500×500	5	Power Iterations	0.062	2005	200	$9.2 \cdot 10^{-3}$	$9.1 \cdot 10^{-3}$
500×500	5	RSVD	0.017	1170	38	$9.8 \cdot 10^{-3}$	$9.6 \cdot 10^{-3}$
500×500	5	TRLan	0.018	131	65	$9.6 \cdot 10^{-5}$	$9.4 \cdot 10^{-5}$
500×500	50	Power Iterations	0.44	43750	437	$9.9 \cdot 10^{-3}$	$9.0 \cdot 10^{-3}$
500×500	50	RSVD	0.61	6120	50	$9.9 \cdot 10^{-3}$	$9.1 \cdot 10^{-3}$
500×500	50	TRLan	0.16	462	231	$3.3 \cdot 10^{-7}$	$3.0 \cdot 10^{-7}$
5000×5000	5	Power Iterations	9.6	9065	906	$8.6 \cdot 10^{-3}$	$8.6 \cdot 10^{-3}$
5000×5000	5	RSVD	2.1	5640	187	$9.7 \cdot 10^{-3}$	$9.7 \cdot 10^{-3}$
5000×5000	5	TRLan	0.70	205	102	$7.7 \cdot 10^{-3}$	$7.7 \cdot 10^{-3}$

Table 2 Newton-Schulz iterations performance on random dense matrices (for reference).

Matrix size	Time (s)	Matvecs	Iterations	err_1
500×500	0.041	27 000	27	$4.8 \cdot 10^{-3}$
5000×5000	26.4	290 000	29	$6.5 \cdot 10^{-3}$

6 Experiments

6.1 A Smooth Convex Problem

We begin by evaluating F-Fanions and S-Fanions on the following L-smooth convex problem:

$$F(\boldsymbol{X}) = \frac{1}{2}\|\boldsymbol{M}^{1/2}\boldsymbol{X}\boldsymbol{N}^{1/2}\|_{\mathrm{F}}^2 = \frac{1}{2}\langle \boldsymbol{X}, \boldsymbol{M}\boldsymbol{X}\boldsymbol{N}\rangle \to \min_{\boldsymbol{X}\in\mathbb{R}^{m\times n}}, \tag{15}$$

where $\boldsymbol{X} \in \mathbb{R}^{m\times n}$ with $m = 500$ and $n = 500$, corresponding to the typical dimensions of a neural network weight matrix, while $\boldsymbol{M} \in \mathbb{S}_+^m$ and $\boldsymbol{N} \in \mathbb{S}_+^n$ are positive-semidefinite matrices. The spectra of $\boldsymbol{M}$ and $\boldsymbol{N}$ are uniformly distributed on the interval $(0, 1)$, so $L \leq 1$ for the Frobenius norm. We initialize $\boldsymbol{X}^0$ with entries drawn independently from $\mathcal{N}(0, 0.01)$.

We evaluate several algorithms from the Fanion family and their convex combinations, using the update rule from (4) with approximate Nesterov momentum:

- **Fanions:** Neon (Fanion-1), Fanion-2, Fanion-10, Fanion-100, and Muon (Fanion-500).
- **F-Fanions:** F-Neon (F-Fanion-1), F-Fanion-2, F-Fanion-10, F-Fanion-100, and F-Muon (F-Fanion-500) with $\alpha = 1/2$.
- **S-Fanions:** S-Neon (S-Fanion-1), S-Fanion-2, S-Fanion-10, S-Fanion-100, and S-Muon (S-Fanion-500) with $\alpha = 1/2$ and `sign_lr_coeff=0.01`.

We also include the baselines of Normalized SGD and SignSGD, which correspond to F-Fanion and S-Fanion, respectively, with $\alpha = 0$ and arbitrary k.

Since theoretical bounds [23, 36] rely on a loose norm bound $\|\cdot\| \leq \rho\|\cdot\|_{\mathrm{F}}$, we do not derive the learning rate or Nesterov momentum from the smoothness constants, which also depend on the choice of the norm. Instead, we identify the `(lr, momentum)` pair for which the optimizer reaches the loss threshold of 0.001 in the fewest iterations. This setting is both realistic and consistent with the corollaries of convergence theorems that propose constant learning rate and momentum coefficients.

We perform a grid search over the following hyperparameter ranges:

- `momentum` $\in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95\}$ for all algorithms,
- `lr` $\in [0.005, 0.020]$ with step 0.001 for Muon, F-Muon, and S-Muon,
- `lr` $\in [5 \cdot 10^{-5}, 2 \cdot 10^{-4}]$ with step $1 \cdot 10^{-5}$ for SignSGD,
- `lr` $\in [0.01, 0.10]$ with step 0.01 for NSGD.

The tuned parameters and the number of iterations required to converge to 0.001 are presented in Table 3. For intermediate-rank Fanions, F-Fanions, and S-Fanions, the hyperparameters `lr`, `momentum`, and `sign_lr_coeff` are set equal to those of Muon, F-Muon, and S-Muon, respectively, as their losses decrease too slowly to reach 0.001 within a reasonable tuning budget.

Table 3 Tuned learning rates and momentum coefficients in the experiments on the smooth convex problem (15).

Algorithm	`lr`	`momentum`	Iterations to 0.001 loss
Muon	0.007	0.5	1060
NSGD	0.08	0.95	1020
F-Muon	0.015	0.7	910
SignSGD	0.016×0.01	0.95	2650
S-Muon	0.011	0.9	890

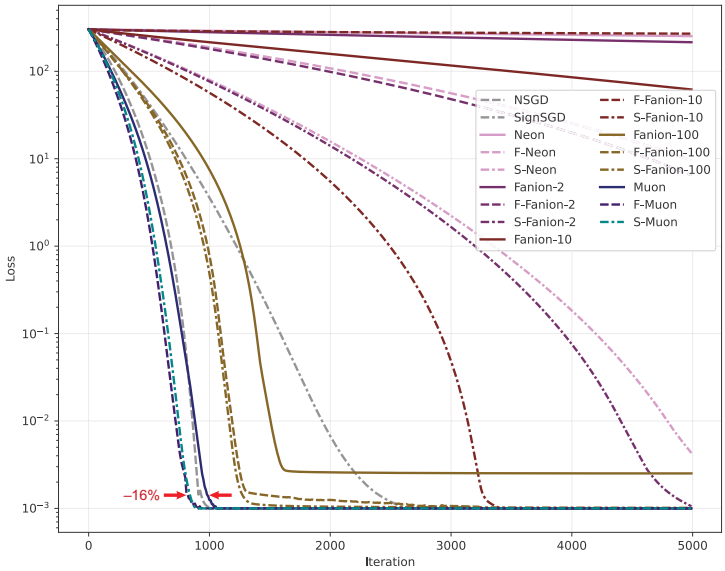
The results are presented in Fig. 1 with additional details in Sect. E.

Both F-Muon and S-Muon converge faster to lower loss values and achieve lower Frobenius norms of the full gradient than NSGD, Muon, or SignSGD.

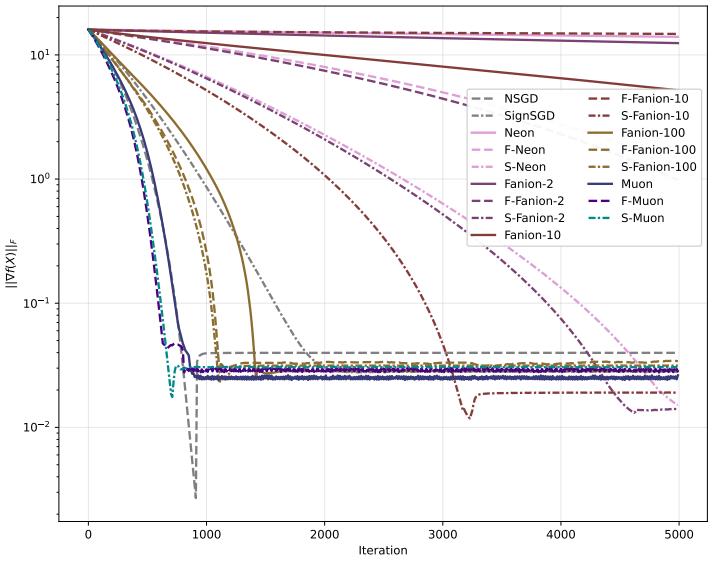
6.2 CIFAR-10 Airbench

We evaluate the algorithms on the CIFAR-10 airbench [21]. To assess the impact of the mixing parameter α in (11), we first run F-Muon for different values of α using hyperparameters tuned for vanilla Muon by Keller Jordan: `lr=0.24(1 - step/total_steps)`, `momentum=0.65`, `nesterov=True` with weight normalization after each weight update. We perform 10 repetitions for each α value and record the accuracy after 8 epochs of training (Fig. 2(a)).

Next, we tune F-Muon specifically with $\alpha = 0.5$, obtaining optimal hyperparameters `lr=0.4(1 - step/total_steps)`, `momentum=0.6`, `nesterov = True`. Tuned F-Muon achieves $94.02 \pm 0.13\%$ validation accuracy after 8 epochs



(a) Loss

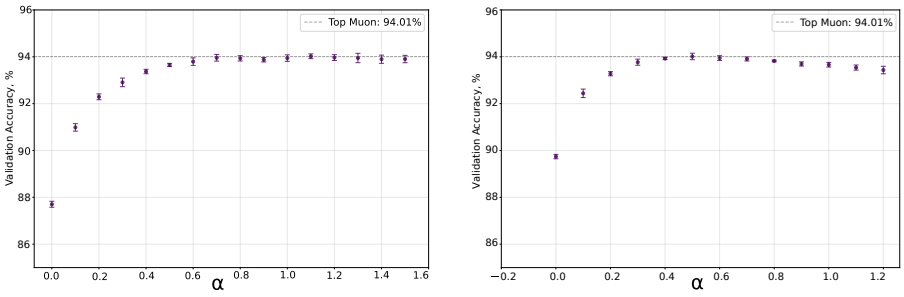


(b) Frobenius norm of the full gradient

Fig. 1 A smooth convex problem (15) for a 500x500 matrix.

(averaged over 200 runs), matching Muon's accuracy and variance. We again measure the validation accuracy as a function of α (Fig. 2(b)) and observe that even at $\alpha = 0.1$, the accuracy substantially exceeds that of vanilla NSGD.

For S-Muon with $\alpha = 0.5$, we tune all hyperparameters to find the optimal configuration `lr=0.42(1 - step/total_steps)`, `momentum=0.63`, `nesterov = True`, `sign_lr_coeff = 0.003`, achieving $94.03 \pm 0.13\%$ validation accuracy (Fig. 2), which slightly exceeds vanilla Muon's performance.



(a) With parameters tuned for Muon (b) With parameters tuned for F-Muon

Fig. 2 Mean validation accuracies for F-Muon with different α.

The Muon-level performance is remarkable given that F-Muon and S-Muon operate with substantially different constraint geometries. Fig. 3 visualizes F-Muon by plotting the LMO balls of Muon and F-Muon (scaled by actual learning rates) in the 2D space of singular values (with all other singular values set to zero). S-Muon cannot be visualized in this manner because the Chebyshev norm is not a function of singular values alone, and visualizing $\|\cdot\|_2$ for $\mathbb{R}^{m\times n}$ with $m, n \geq 2$ requires at least a 4D space. Nevertheless, the LMO ball of S-Muon clearly differs substantially from Muon's, as the effective SignSGD learning rate $\texttt{lr} = (1-\alpha)\texttt{sign\_lr\_coeff} = 6.3 \cdot 10^{-4}$ is comparable to typical SignSGD learning rates in deep learning.

Notably, even the pathological case $\alpha > 1$, which corresponds to an LMO constraint region that is not a norm ball, achieves accuracy nearly identical to vanilla Muon. These observations raise a fundamental question about the sensitivity of LMO-based algorithms to the shape of the constraint region.

Additional results, including gradient norm profiling and performance comparisons with Fanions and F-Fanions, are provided in Sect. F.

6.3 NanoGPT Speedrun

We evaluate F-Muon and S-Muon with $\alpha = 0.5$ against Muon, SignSGD, and NSGD on the NanoGPT speedrun benchmark [18]. The optimal hyperparameters are shown in the legend of Fig. 4, with `sign_sgd_coeff` values of $3 \cdot 10^{-4}$ for S-Muon. After 1750 training steps, F-Muon achieves a cross-entropy loss

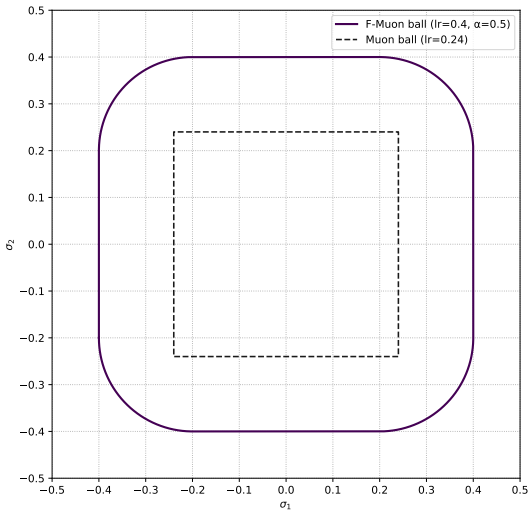


Fig. 3 The LMO balls of Muon and F-Muon for training a CNN on CIFAR-10.

of 3.281, S-Muon achieves 3.287, and Muon achieves 3.279, falling below the target threshold of 3.280. As illustrated in Fig. 4, these differences are negligible. Notably, F-Muon, a convex combination of Muon and NSGD, is efficient despite NSGD performing poorly in isolation. The same observation holds for S-Muon and SignSGD.

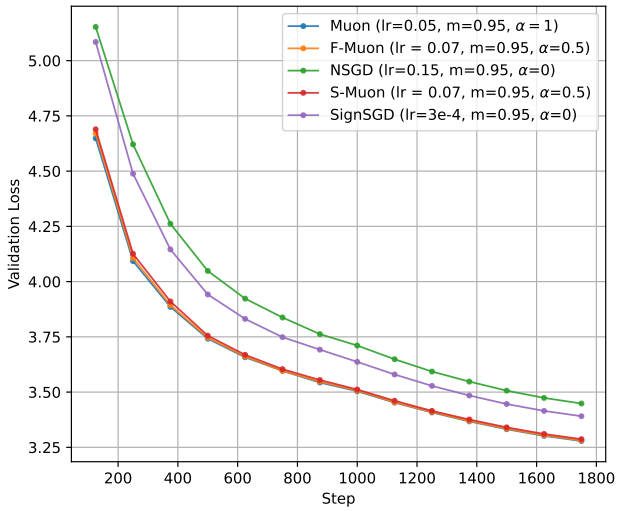


Fig. 4 Validation loss for NanoGPT.

As observed in the CIFAR-10 airbench experiments, setting $\alpha = 1.1$ for F-Muon (corresponding to the no-ball configuration) yields a loss of 3.2818, representing only a marginal difference from the baseline.

Interestingly, F-Muon with the NSGD component *not scaled* by Muon's $\sqrt{m/n}$ rule yields a loss of 3.288 (an increase of 0.007), while S-Muon *scaled* with this rule achieves a loss of 3.292 (an increase of 0.005). Thus, there is no clear rule as to whether the non-Fanion update part should be scaled by $\sqrt{m/n}$.

6.4 GPT-2 Medium Speedrun

We scale from NanoGPT to GPT-2 Medium (24 transformer layers, 1024-dimensional hidden layers, 16 attention heads, approximately 345 million parameters), evaluating the same algorithms with $\alpha = 0.5$ on the FineWeb dataset. After 5960 training steps, Muon achieves a validation loss of 2.9198, successfully reaching the speedrun threshold of 2.92. F-Muon achieves 2.9215, and S-Muon achieves 2.9235, narrowly missing the threshold. As illustrated in Fig. 5, the performance gap remains remarkably small: F-Muon trails Muon by only 0.0017, while S-Muon trails by 0.0037. These minimal differences demonstrate that alternative norm constraints maintain competitive performance even at this significantly larger model scale.

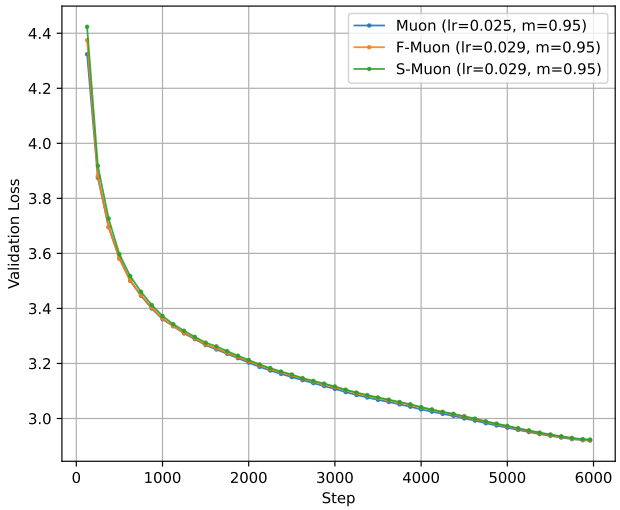


Fig. 5 Validation loss for GPT-2 Medium.

6.5 NanoGPT Fine-Tuning

We fine-tuned the NanoGPT framework by Karpathy [20] using the standard GPT-2 Medium configuration. We selected the *TinyStories* corpus [10] as the training dataset for its high entropy and structural diversity, which amplify the differences in optimization dynamics. All training runs were initialized with the same random seed, weight initialization, and learning rate schedule to ensure that performance differences arose solely from the choice of the optimizer.

For all layers with one-dimensional outputs, we used AdamW with a learning rate of $1 \cdot 10^{-3}$. Since momentum exhibited negligible influence on training dynamics, it was held constant across all experiments. Fig. 6 presents a comparative analysis of optimization performance. Notably, F-Muon is significantly more robust to the choice of learning rate than Muon and S-Muon.

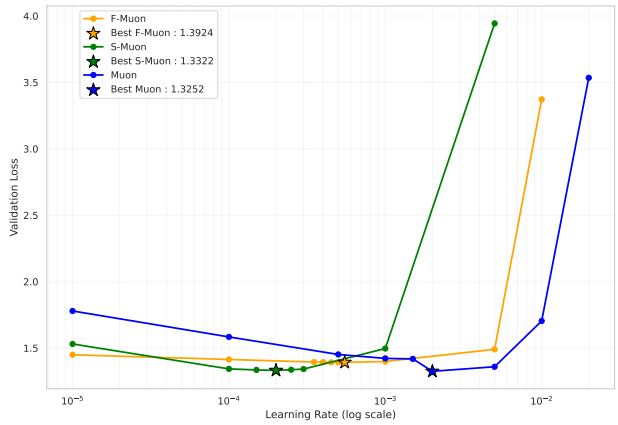


Fig. 6 Comparison of validation loss for Muon, F-Muon, and S-Muon across a range of learning rates. Stars denote the best learning rates for each optimizer. $\alpha = 1/2$ for F-Muon and S-Muon and `sign_sgd_coeff` $= 1/3$ for S-Muon.

Fig. 7 presents the training and validation loss curves at the optimal learning rate for each optimizer. While F-Muon and S-Muon maintain consistent behavior, vanilla Muon achieves the lowest loss overall, outperforming other algorithms by a small margin.

7 Related Work and Discussion

7.1 Algorithms for Vectors → Algorithms for Matrices

The updates of LMO optimizers exhibit striking similarities between vector l_p norms and matrix Schatten S_p norms, as illustrated in Table 4. These parallels extend beyond the update formulas themselves to empirical performance characteristics. SignSGD is related to Adam, as noted in [4], and both SignSGD

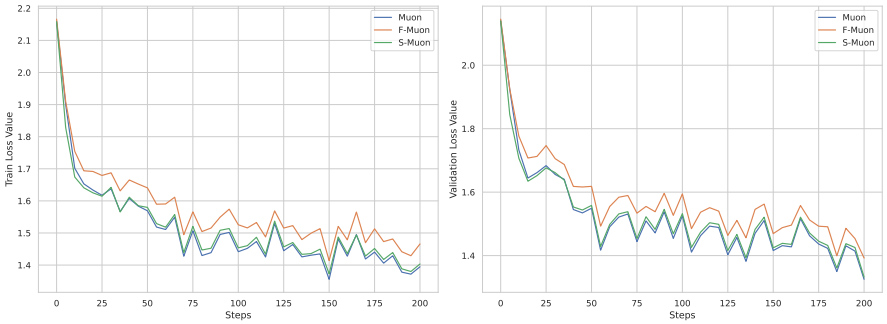


Fig. 7 Train and validation loss at the optimal learning rate for each optimizer.

and Muon perform well during training large models [45, 27]. NSGD maintains identical updates in both vector and matrix cases. Greedy Coordinate Descent can hardly be applied to high-dimensional problems, which provides a perspective on why one-rank Neon underperforms in such settings.

Table 4 LMO optimizers in Schatten S_p norms and in l_p norms. g is the gradient. When it is a matrix, $g = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$.

Algorithm	LMO constraint set $\mathcal{D}$	LMO	Reference
Normalized SGD	l_2-ball, S_2-ball	$-\eta \frac{g}{\|g\|_2} = -\eta \frac{g}{\|g\|_\mathrm{F}}$	[16]
Momentum Normalized SGD	Ball in l_2, or Ball in S_2	$-\eta \frac{g}{\|g\|_2} = -\eta \frac{g}{\|g\|_\mathrm{F}}$	[9]
SignSGD	Ball in Max-norm l_∞	$-\eta\,\mathrm{sign}(g)$	[5, Thm. 1]
Signum	Ball in Max-norm l_∞	$-\eta\,\mathrm{sign}(g)$	[5, Thm. 3]
Muon	Ball in Spectral S_∞	$-\eta \boldsymbol{U}\boldsymbol{V}^\top$	[19]
Gauss-Southwell Coordinate Descent	Ball in l_1	$-\eta \sum_{i \in \arg\max \lvert g_i^t \rvert} \mathrm{sign}(g_i^t) e_i$	[38, p. 19]
Neon	Ball in Nuclear S_1	$-\eta \boldsymbol{u}_1 \boldsymbol{v}_1^\top$	This work

7.2 Theory behind Muon

Since Muon [19] is a highly efficient optimizer for functions of weight matrices, substantial research has focused on two objectives: further improving its performance and theoretically explaining its success.

There has been a prolonged gap in the theoretical understanding of Muon, excluding the simplistic derivation of Muon [3] that was based on [4]. This gap, in our view, remains incompletely closed. For example, Kovalev [23] provided convergence guarantees for Muon in various settings, from which, however, Muon's empirical supremacy cannot be recovered. Indeed, although the obtained bounds depend on the norm choice, the convergence asymptotics remain the same as for NSGD and other optimizers: $K = \mathcal{O}(\varepsilon^{-4})$ in the L-smooth stochastic case.

A similar limitation affects [36], where the L-smoothness assumption is replaced with a more practical (L_0, L_1)-smoothness. By estimating smoothness, the authors recovered the optimal fine-tuned step sizes reported by [32]. However, they have not theoretically demonstrated the optimality of the spectral or RMS-to-RMS norm, which is observed in practice, as our comparison with NSGD and Neon highlights.

A common limitation of these analyses is their focus on convergence measured by the gradient norm. As we showed in our CIFAR experiments (Sect. F), the stochastic gradient norm may decrease by only a factor of ten when the training accuracy reaches 100%.

Another way to understand Muon is to focus on particular problems, like multi-class linear classification on separable data [11]. Curiously, Neon has the same margin convergence rate as Muon because both are based on Schatten norms. That being said, the norms for margins are different: Muon has an implicit bias toward solutions that minimize the spectral norm of the weight matrix, while Neon tends to minimize its nuclear norm.

We hypothesize that the stark performance discrepancy between Neon and Muon, both of which are described by the Stochastic Conditional Gradient method [32] or Gluon [36] frameworks, lies in the structure of the norm ball or in the preconditioner interpretation of the algorithm [30], which warrants further investigation.

7.3 Improvements of Muon → of Fanions, F-Fanions, and S-Fanions as well

A large number of applications and improvements of Muon have been proposed in less than a year. Liu et al. [27] adapted the algorithm for training language models larger than NanoGPT. Shah et al. [37] enabled efficient hyperparameter transfer by combining Muon with maximal update parametrization. To construct their COSMOS optimizer, Chen et al. [8] applied computationally intensive updates of the SOAP optimizer [42] to a low-dimensional "leading eigensubspace" while using memory-efficient methods like Muon for the remaining parameters. Amsel et al. [2] and Grishina et al. [12] proposed more efficient alternatives to tuning the coefficients in the Newton-Schulz algorithm. Si et al. [40], Veprikov et al. [41], and Li et al. [26] introduced AdaMuon, MuAdam, and NorMuon, respectively, which combine gradient orthogonalization with adaptivity.

Fanios, F-Fanions, and S-Fanions benefit from these improvements and many others. They could be transformed into Drop-Fanions by updating only the selected layers, as in [14]. They can be viewed as approximations of the Non-Euclidean Proximal Point Method for the corresponding norms [15]. They can be clipped to produce ClippedScion-like algorithms [33]. They could be made more memory-efficient through the zero-order techniques that proved effective for Muon [34]. Finally, the results from [39] can be used to explain the robustness of Muon to the norm changes observed in our experiments and to theoretically derive faster yet effective approximate schemes for calculating

the LMO; power iterations with a limited number of iterations represent a promising direction for this analysis.

7.4 The Nuclear Norm in the LMO

We discovered during the preparation of this article that the nuclear norm has already been explored in the context of the linear minimization oracle. Pethick et al. [31] applied it to create νSAM, a novel sharpness-aware minimization technique. It would be interesting to substitute $\|\cdot\|_*$ with $\|\cdot\|^\dagger_{\text{KF-}k}$ in their approach. Since the SAM neighborhood becomes more diverse, using $k > 1$ might enhance the accuracy boost from using SAM while preserving a small memory footprint and minimal time overhead if Dion-style power iterations are employed.

7.5 The LMO and Error Feedback

As previously mentioned, rank-k unsharded Dion without error feedback and update scaling is equivalent to Fanion-k. Since error feedback is crucial for Dion, as demonstrated by the ablation study in [1], F-Fanions and S-Fanions would benefit from it as well. In federated learning, error feedback proves effective even for compressed Muon [13]. Fanions and S-Fanions offer a transmission advantage, requiring fewer bits: $\sum_{i=1}^k \boldsymbol{u}_i \boldsymbol{v}_i^\top$ can be efficiently transmitted as $\{\boldsymbol{u}_1, \boldsymbol{v}_1, \ldots, \boldsymbol{u}_k, \boldsymbol{v}_k\}$ ($(m+n) \times k$ floats), while the sign component can be encoded in $m \times n$ bits. Thus, compression is inherently built into the representation. Moreover, there is an intriguing possibility to construct differentially private Fanions and S-Fanions using more optimal non-Gaussian noise, as was done with DP-SignSGD [17]. We leave this for future research.

8 Conclusions

In this article, we addressed the central question of whether one should constrain by the spectral or any other operator norm in deriving Muon-like updates, and how alternative norms affect performance and computational cost. Our answer is that the choice of the matrix norm is remarkably flexible: properly tuned variants based on alternative norms can match or even slightly exceed Muon's performance on real-world tasks, while offering additional benefits such as improved learning rate robustness.

We generalized several successful algorithms to linear minimization oracle (LMO) based algorithms using the family of norms dual to Ky Fan k-norms, yielding the Fanion family with low-rank updates. We further proposed the technique of combining them with Normalized SGD or SignSGD, creating the F-Fanion and S-Fanion families. Our experiments demonstrate that F-Muon and S-Muon achieve competitive performance with Muon on CIFAR-10

airbench and NanoGPT speedrun tasks, confirming that the underlying norm constraint can be significantly modified without sacrificing effectiveness.

However, our results also reveal that not every LMO-based algorithm is effective: Neon (rank-one Fanion) underperforms despite sharing the same theoretical convergence asymptotics as Muon in existing bounds. We suggest that future work on non-Euclidean LMO algorithms should explain the observed empirical superiority of Muon over other Fanions in the corollaries to the convergence theorems, and ideally account for the robustness of F-Muon and S-Muon as well.

Declarations

Funding The authors did not receive support from any organization for the submitted work.
Competing Interests The authors have no relevant financial or non-financial interests to disclose.
Data Availability No datasets were generated during the current study. All data used in the experiments is publicly available.
Author Contributions IO suggested using the nuclear norm in the Bernstein–Newhouse framework [4]. DM presented the problem at the MIPT optimization course, supervised the project, and helped to revise the manuscript. IK suggested using composite norms (though not the ones that induce Fanions, F-Fanions, or S-Fanions) and helped to draft and revise the manuscript. NK suggested the Lanczos algorithm as the most precise means to compute Fanions' updates, conducted experiments to prove this, and wrote Sect. 5. AV conducted the finetuning of NanoGPT on TinyStories and wrote Sect. 6.5. All other work was done by AK: constructing Fanions, F-Fanions, and S-Fanions, conducting experiments on the smooth convex problem and on the CIFAR-10 airbench, pretraining NanoGPT and GPT-2 Medium, and writing the manuscript.

Acknowledgements We thank Dmitry Kovalev and Egor Shulgin for helpful discussions.

References

1. Ahn, K., Xu, B., Abreu, N., Fan, Y., Magakyan, G., Sharma, P., Zhan, Z., Langford, J.: Dion: Distributed orthonormalized updates. arXiv preprint arXiv:2504.05295 (2025)
2. Amsel, N., Persson, D., Musco, C., Gower, R.M.: The Polar Express: Optimal matrix sign methods and their application to the Muon algorithm. arXiv preprint arXiv:2505.16932 (2025)
3. Bernstein, J.: Deriving Muon (2025). URL https://jeremybernste.in/writing/deriving-muon

4. Bernstein, J., Newhouse, L.: Old optimizer, new norm: An anthology. arXiv preprint arXiv:2409.20325 (2024)
5. Bernstein, J., Wang, Y.X., Azizzadenesheli, K., Anandkumar, A.: signSGD: Compressed optimisation for non-convex problems. In: International conference on machine learning, pp. 560–569. PMLR (2018)
6. Bhatia, R.: Matrix analysis, vol. 169. Springer Science & Business Media (2013)
7. Cesista, F.L.: Steepest descent under Schatten-p norms (2025). URL https://leloykun.github.io/ponder/steepest-descent-schatten-p/
8. Chen, W., Liang, C., Zhao, T., Zhang, Z., Kang, H., Liu, L., Li, Z., Xu, Z.: COSMOS: A hybrid adaptive optimizer for memory-efficient training of LLMs. arXiv preprint arXiv:2502.17410 (2025)
9. Cutkosky, A., Mehta, H., Orabona, F.: Momentum-based variance reduction in nonconvex SGD. Advances in Neural Information Processing Systems (2020)
10. Eldan, R., Li, Y.: TinyStories: How small can language models be and still speak coherent English? arXiv preprint arXiv:2305.07759 (2023)
11. Fan, C., Schmidt, M., Thrampoulidis, C.: Implicit bias of spectral descent and Muon on multiclass separable data. arXiv preprint arXiv:2502.04664 (2025)
12. Grishina, E., Smirnov, M., Rakhuba, M.: Accelerating Newton-Schulz iteration for orthogonalization via Chebyshev-type polynomials. arXiv preprint arXiv:2506.10935 (2025)
13. Gruntkowska, K., Gaponov, A., Tovmasyan, Z., Richtárik, P.: Error feedback for Muon and friends. arXiv preprint arXiv:2510.00643 (2025)
14. Gruntkowska, K., Maziane, Y., Qu, Z., Richtárik, P.: Drop-Muon: Update less, converge faster. arXiv preprint arXiv:2510.02239 (2025)
15. Gruntkowska, K., Richtárik, P.: Non-Euclidean broximal point method: A blueprint for geometry-aware optimization. arXiv preprint arXiv:2510.00823 (2025)
16. Hazan, E., Levy, K., Shalev-Shwartz, S.: Beyond convexity: Stochastic quasi-convex optimization. Advances in neural information processing systems **28** (2015)
17. Jang, J., Hwang, S., Yang, H.J.: Rethinking DP-SGD in discrete domain: Exploring logistic distribution in the realm of signSGD. In: R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, F. Berkenkamp (eds.) Proceedings of the 41st International Conference on Machine Learning, *Proceedings of Machine Learning Research*, vol. 235, pp. 21,241–21,265. PMLR (2024)
18. Jordan, K., Bernstein, J., Rappazzo, B., @fernbear.bsky.social, Vlado, B., Jiacheng, Y., Cesista, F., Koszarsky, B., @Grad62304977: moddednanogpt: Speedrunning the NanoGPT baseline (2024). URL https://github.com/KellerJordan/modded-nanogpt
19. Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., Bernstein, J.: Muon: An optimizer for hidden layers in neural networks (2024). URL

https://kellerjordan.github.io/posts/muon/
20. Karpathy, A.: nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. GitHub repository (2022). URL https://github.com/karpathy/nanoGPT
21. Keller, J.: CIFAR-10 airbench (2023). URL https://github.com/KellerJordan/cifar10-airbench
22. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980 (2014)
23. Kovalev, D.: Understanding gradient orthogonalization for deep learning via non-Euclidean trust-region optimization. arXiv preprint arXiv:2503.12645 (2025)
24. Kovalev, D., Borodich, E.: Non-Euclidean SGD for structured optimization: Unified analysis and improved rates. arXiv preprint arXiv:2511.11466 (2025)
25. Li, J., Hong, M.: A note on the convergence of Muon. arXiv preprint arXiv:2502.02900 (2025)
26. Li, Z., Liu, L., Liang, C., Chen, W., Zhao, T.: NorMuon: Making Muon more efficient and scalable. arXiv preprint arXiv:2510.05491 (2025)
27. Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., et al.: Muon is scalable for LLM training. arXiv preprint arXiv:2502.16982 (2025)
28. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101 (2017)
29. Pethick, T.: Understanding Dion (2025). URL https://pethick.dk/posts/2025-08-18-understanding-dion/
30. Pethick, T., Antonakopoulos, K., Silveti-Falls, A., Vankadara, L.C., Cevher, V.: Training neural networks at any scale. arXiv preprint arXiv:2511.11163 (2025)
31. Pethick, T., Raman, P., Minorics, L., Hong, M., Sabach, S., Cevher, V.: νSAM: Memory-efficient sharpness-aware minimization via nuclear norm constraints. Transactions on Machine Learning Research (2025)
32. Pethick, T., Xie, W., Antonakopoulos, K., Zhu, Z., Silveti-Falls, A., Cevher, V.: Training deep learning models with norm-constrained LMOs. In: Forty-second International Conference on Machine Learning (2025)
33. Pethick, T., Xie, W., Erdogan, M., Antonakopoulos, K., Silveti-Falls, A., Cevher, V.: Generalized gradient norm clipping & non-Euclidean (L_0, L_1)-smoothness. Advances in Neural Information Processing Systems **38**, 21,170–21,208 (2026)
34. Petrov, E., Grigoriy, E., Antonov, A., Veprikov, A., Plyusnin, P., Bushkov, N., Moiseev, S., Beznosikov, A.: Leveraging coordinate momentum in SignSGD and Muon: Memory-optimized zero-order LLM fine-tuning. In: Tiny Titans: The next wave of On-Device Learning for Foundational Models (TTODLer-FM) (2025)
35. Preferred Infrastructure, Inc., Developers, C.: CuPy: `cupyx.scipy.sparse.linalg.svds` — api reference. https://docs.cupy.dev/en/stable/reference/generated/cupyx.scipy.

sparse.linalg.svds.html (2025). Accessed: 2025-08-24
36. Riabinin, A., Shulgin, E., Gruntkowska, K., Richtárik, P.: Gluon: Making Muon & Scion great again! (bridging theory and practice of LMO-based optimizers for LLMs). arXiv preprint arXiv:2505.13416 (2025)
37. Shah, I., Polloreno, A.M., Stratos, K., Monk, P., Chaluvaraju, A., Hojel, A., Ma, A., Thomas, A., Tanwer, A., Shah, D.J., et al.: Practical efficiency of Muon for pretraining. arXiv preprint arXiv:2505.02222 (2025)
38. Shi, H.J.M., Tu, S., Xu, Y., Yin, W.: A primer on coordinate descent algorithms. arXiv preprint arXiv:1610.00040 (2016)
39. Shulgin, E., AlRashed, S., Orabona, F., Richtárik, P.: Beyond the ideal: Analyzing the inexact Muon update. arXiv preprint arXiv:2510.19933 (2025)
40. Si, C., Zhang, D., Shen, W.: AdaMuon: Adaptive Muon optimizer. arXiv preprint arXiv:2507.11005 (2025)
41. Veprikov, A., Bolatov, A., Horváth, S., Beznosikov, A., Takáč, M., Hanzely, S.: Preconditioned norms: A unified framework for steepest descent, quasi-Newton and adaptive methods. arXiv preprint arXiv:2510.10777 (2025)
42. Vyas, N., Morwani, D., Zhao, R., Shapira, I., Brandfonbrener, D., Janson, L., Kakade, S.: SOAP: Improving and stabilizing Shampoo using Adam for language modeling. In: International Conference on Learning Representations, vol. 2025, pp. 93,423–93,444 (2025)
43. Wu, K., Simon, H.: Thick-Restart Lanczos method for large symmetric eigenvalue problems. SIAM Journal on Matrix Analysis and Applications **22**(2), 602–616 (2000). DOI 10.1137/S0895479898334605
44. Yu, Y.L.: Arithmetic duality for norms (2012). URL https://cs.uwaterloo.ca/~y328yu/notes/normduality.pdf
45. Zhao, R., Morwani, D., Brandfonbrener, D., Vyas, N., Kakade, S.: Deconstructing what makes a good optimizer for autoregressive language models. In: International Conference on Learning Representations, vol. 2025, pp. 2830–2850 (2025)

A LMO for Neural Networks

In a typical neural network, the objective function F depends on a set of weight matrices $\{\boldsymbol{W}_1, \boldsymbol{W}_2, \ldots\}$. The optimization framework we have described is applied in a layer-wise fashion. At each iteration t, a stochastic gradient $g(\boldsymbol{X}^t, \xi^t)$ is computed using a mini-batch of data with the ξ^t noise via backpropagation. This yields a separate gradient component, $\boldsymbol{G}_i^t$, for each matrix $\boldsymbol{X}_i$. The LMO-based update rule is then applied to each matrix $\boldsymbol{X}_i$ using its corresponding gradient component $\boldsymbol{G}_i^t$.

B Proof of Lemma on the Dual to Convex Combinations

We provide here the proof of Lemma 4.1.

Proof Let us first prove the lemma for the case $n = 2$. Denote $f(x) = \alpha_1 \|x\|_{(1)}$ and $g(x) = \alpha_2 \|x\|_{(2)}$, such that $\|x\| = f(x) + g(x)$. Recall two standard facts:

1. For any norm $\|\cdot\|$ and $\lambda > 0$,
$$(\lambda\|\cdot\|)^*(y) = \sup_x \big(\langle y, x\rangle - \lambda\|x\|\big) = \delta_{\lambda\mathcal{B}_{\|\cdot\|^\dagger}}(y)\,,$$
i.e., the indicator function of the scaled dual ball.
2. The Fenchel conjugate of a sum satisfies
$$(f+g)^*(y) = \inf_{u+v=y}\big(f^*(u) + g^*(v)\big)\,.$$

Applying these to f and g, we have
$$f^*(u) = \delta_{\alpha_1\mathcal{B}_{\|\cdot\|^\dagger_{(1)}}}(u)\,, \quad g^*(v) = \delta_{\alpha_2\mathcal{B}_{\|\cdot\|^\dagger_{(2)}}}(v)\,.$$

Thus,
$$\|\cdot\|^*(y) = (f+g)^*(y) = \inf_{u+v=y}\big(\delta_{\alpha_1\mathcal{B}_{\|\cdot\|^\dagger_{(1)}}}(u) + \delta_{\alpha_2\mathcal{B}_{\|\cdot\|^\dagger_{(2)}}}(v)\big) = \delta_{\alpha_1\mathcal{B}_{\|\cdot\|^\dagger_{(1)}} + \alpha_2\mathcal{B}_{\|\cdot\|^\dagger_{(2)}}}(y).$$

By definition, the conjugate of a norm is exactly the indicator of its dual unit ball:
$$\|\cdot\|^*(y) = \delta_{\mathcal{B}_{\|\cdot\|^\dagger}}(y)\,.$$

Therefore, $\mathcal{B}_{\|\cdot\|^\dagger} = \alpha_1\mathcal{B}_{\|\cdot\|^\dagger_{(1)}} + \alpha_2\mathcal{B}_{\|\cdot\|^\dagger_{(2)}}$.

Now we prove the general case by induction. The base case ($n = 2$) is already proven. Suppose that the assumption of the lemma holds for $n = k$. Then, for $n = k+1$,
$$\|x\| = \sum_{i=1}^{k}\alpha_i\|x\|_{(i)} + \alpha_{k+1}\|x\|_{(k+1)} = \|x\|_{(1:k)} + \alpha_{k+1}\|x\|_{(k+1)}\,.$$

Applying the result for $n = 2$ combined with the induction assumption, we obtain
$$\mathcal{B}_{\|\cdot\|^\dagger} = \mathcal{B}_{\|\cdot\|^\dagger_{(1:k)}} + \alpha_{k+1}\mathcal{B}_{\|\cdot\|^\dagger_{(k+1)}} = \sum_{i=1}^{k+1}\alpha_i\mathcal{B}_{\|\cdot\|^\dagger_{(i)}}\,,$$

which proves the lemma. □

C Norms $\|\cdot\|^\dagger_{\mathbf{F}*}$ and $\|\cdot\|^\dagger_{\mathbf{F2}}$

Based on Lemma 4.1, we immediately find $\|\cdot\|^\dagger_{\mathrm{F}*}$, which is related to the F-Muon update. Indeed, after setting $\beta = 1-\alpha$ and remembering that for smooth and bounded cases we can use min instead of inf, we obtain
$$\|\boldsymbol{Y}\|^\dagger_{\mathrm{F}*} = \min_{\boldsymbol{Z}}\min_t\{t, s.t. \|\boldsymbol{Z}\|_2 \le \alpha t, \|\boldsymbol{Y}-\boldsymbol{Z}\|_{\mathrm{F}} \le (1-\alpha)t\}\,. \tag{16}$$

If $\alpha = 1$, then $\boldsymbol{Z} = \boldsymbol{Y}$, and we get $\|\boldsymbol{Y}\|^\dagger_{\mathrm{F}*} = \|\boldsymbol{Y}\|_2$. If $\alpha = 0$, then $\boldsymbol{Z} = 0$, and we get $\|\boldsymbol{Y}\|^\dagger_{\mathrm{F}*} = \|\boldsymbol{Y}\|_{\mathrm{F}}$.

Similarly, we find $\|\cdot\|^\dagger_{\mathrm{F2}}$, which is related to the F-Neon update:
$$\|\boldsymbol{Y}\|^\dagger_{\mathrm{F2}} = \min_{\boldsymbol{Z}}\min_t\{t, s.t. \|\boldsymbol{Z}\|_* \le \alpha t, \|\boldsymbol{Y}-\boldsymbol{Z}\|_{\mathrm{F}} \le (1-\alpha)t\}\,. \tag{17}$$

If $\alpha = 1$, then $\boldsymbol{Z} = \boldsymbol{Y}$, and we get $\|\boldsymbol{Y}\|^\dagger_{\mathrm{F2}} = \|\boldsymbol{Y}\|_*$. If $\alpha = 0$, then $\boldsymbol{Z} = 0$, and we get $\|\boldsymbol{Y}\|^\dagger_{\mathrm{F2}} = \|\boldsymbol{Y}\|_{\mathrm{F}}$.

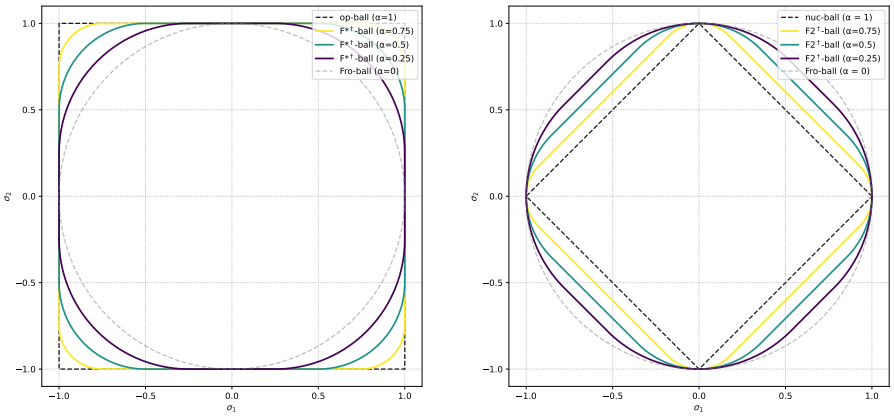
Unfortunately, $\|\cdot\|^\dagger_{\mathrm{F}*}$ and $\|\cdot\|^\dagger_{\mathrm{F2}}$ do not have simple closed-form expressions and cannot be computed as easily as their duals.

D Visualization of Different Matrix Norms

D.1 Duals to F* and F2 Norms

It follows from Lemma 4.1 that the norm ball in $\|\cdot\|^\dagger_{\mathrm{F}*}$ is the Minkowski sum of the norm ball in $\alpha\|\cdot\|_*$ and $(1-\alpha)\|\cdot\|_{\mathrm{F}}$, and the norm ball in $\|\cdot\|^\dagger_{\mathrm{F2}}$ is the Minkowski sum of the norm ball in $\alpha\|\cdot\|_2$ and $(1-\alpha)\|\cdot\|_{\mathrm{F}}$.

In Fig. 8, we plot these norms. The x-axis and y-axis represent the singular values σ_1 and σ_2, respectively, of a matrix from $\mathbb{R}^{m\times n}$ with $\min\{m,n\} = 2$.



(a) LMO balls for F-Muon for different α (b) LMO balls for F-Neon for different α

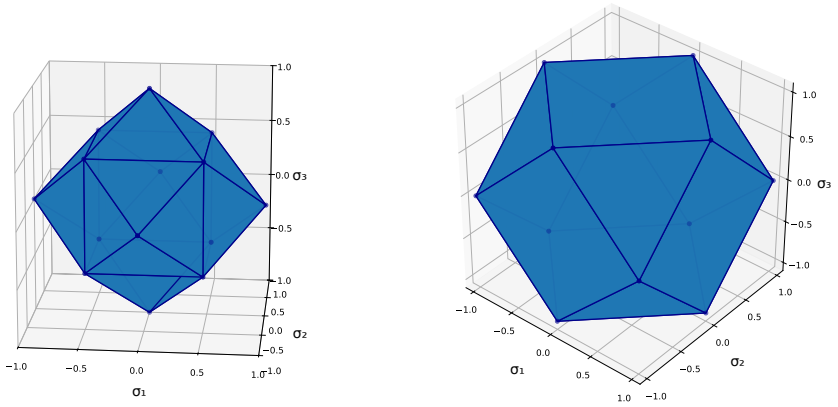
Fig. 8 Balls in the duals to F* and F2 norms for different α.

D.2 A Ky Fan Norm and Its Dual

While 1-balls in l_∞, l_1, and l_2 norms are well-known from textbooks, the Ky Fan k-norm presents a more intricate structure.

To showcase the complex geometry of the Ky Fan k-norm and its dual, we present Fig. 9, which displays the unit ball in the Ky Fan 2-norm (Fig. 9(a)) and its dual (Fig. 9(b)). The x-, y-, and z-axes represent the singular values σ_1, σ_2, and σ_3, respectively, of a matrix from $\mathbb{R}^{m\times n}$ with $\min\{m,n\} = 3$. In this representation, we do not sort the singular values. We plot unit balls in the Top-2 norm $\max\{|x|+|y|, |x|+|z|, |y|+|z|\}$ and its dual norm $\max\{\max(|x|,|y|,|z|), (|x|+|y|+|z|)/2\}$. The resulting balls exhibit significantly greater complexity than those in l_∞, l_1, and l_2 norms.

These balls can be described more simply using the results from Yu [44]. The Ky Fan 2-norm ball is an intersection of three l_1 balls in (x,y), (x,z), and (y,z) spaces. The unit ball in the dual Ky Fan 2-norm is an intersection of the unit ball in the l_∞ norm and the 1/2-ball in the l_1 norm.



(a) The Ky Fan 2-norm (b) The dual of the Ky Fan 2-norm

Fig. 9 The Ky Fan 2-norm and its dual in the space of three singular values.

E More Details for the Smooth Convex Problem

The spectral and nuclear norms of the full gradients over iterations, as well as the loss over time, are shown in Fig. 10. The poor performance of Fanions and F-Fanions in terms of speed for large k is likely caused by an inefficient implementation of TRLan.

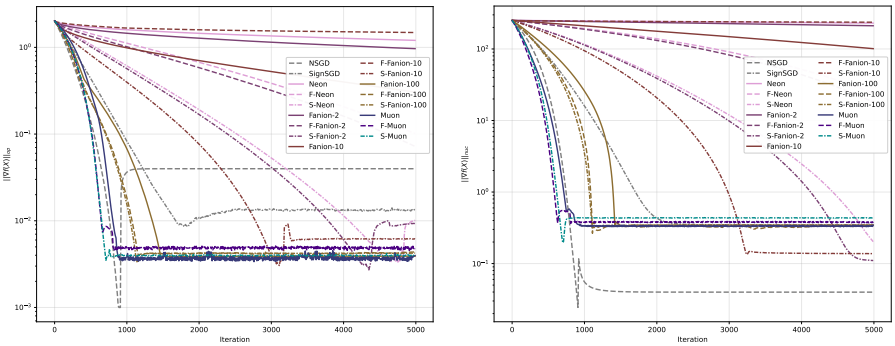
F Gradient Norm Analysis for CIFAR-10 Experiments

Theoretical convergence bounds for Muon and other LMO-based algorithms are typically expressed in terms of gradient norms [25, 23, 32, 36, 24]. Accordingly, we measure these norms during training on a real deep learning problem to validate theoretical predictions.

Table 5 Parameters for CIFAR-10 airbench. `sign_lr_mult` for S-Muon is 0.003.

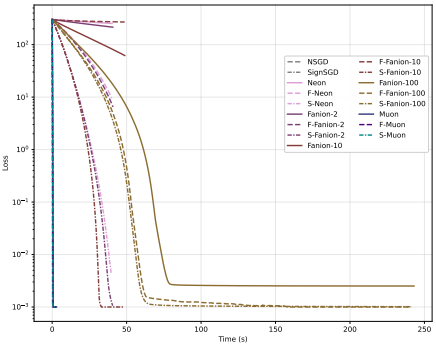
Method	`lr`	`momentum`	val_accuracy, %
NSGD	0.5	0.95	91.6 ± 0.52
SignSGD	0.003	0.95	91.54 ± 0.26
Muon	0.24	0.6	94.01 ± 0.10
F-Muon	0.40	0.6	94.01 ± 0.13
S-Muon	0.42	0.63	94.03 ± 0.13
Neon	0.24	0.6	69.8 ± 0.5
F-Neon	0.40	0.6	87.15 ± 0.24
Fanion-5	0.24	0.6	80.69 ± 1.25
F-Fanion-5	0.40	0.6	86.66 ± 0.65

We compare Muon, F-Muon, S-Muon, NSGD, SignSGD, Neon, F-Neon, Fanion-5, and F-Fanion-5 on CIFAR-10 airbench. NSGD and SignSGD are not heavily tuned. The hyperparameters of Neon, F-Neon, Fanion-5, and F-Fanion-5 are taken from Muon and F-Muon



(a) Spectral norm of the full gradient

(b) Nuclear norm of the full gradient



(c) Loss over time

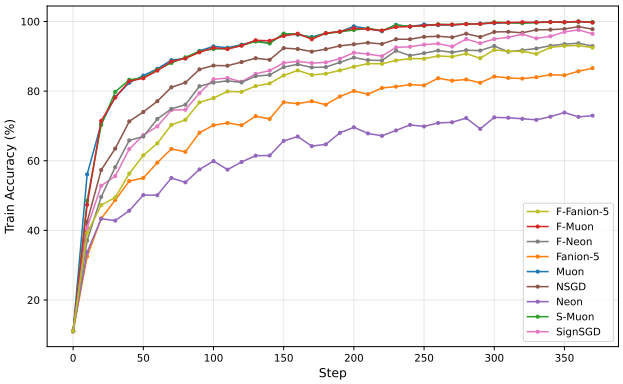
Fig. 10 More plots for the smooth convex problem (15) for a 500x500 matrix.

(see Table 5). The validation accuracies reported after 8 epochs correspond to the airbench variant with weight normalization applied at each step. However, to remain faithful to the conditions assumed in convergence theorems [23, 36], we do not normalize network weights later when logging gradient norms.

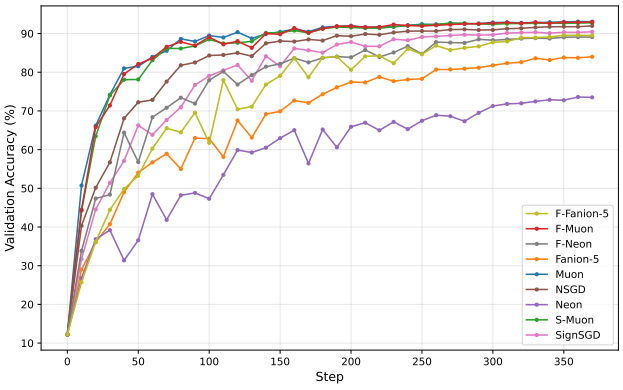
We log train and validation accuracies (Fig. 11), as well as the Frobenius, spectral, and nuclear norms of the gradients for `conv1.weight` and `conv2.weight` across all layers (see Fig. 12 for norms in layer 2, and Fig. 13 for total gradient norms). During training, gradient norms decrease by at most one order of magnitude, while training accuracy reaches 100% for Muon, S-Muon, and F-Muon. This observation suggests that convergence bounds of the form $\max\{\frac{A}{\varepsilon}, \frac{B}{\varepsilon^2}, \frac{C}{\varepsilon^3}, \frac{D}{\varepsilon^4}\}$ (such as Corollary 2 from Kovalev [23]) should not be simplified to $\frac{D}{\varepsilon^4}$ when selecting an optimal algorithm for practical applications: final ε may be quite large.

G Technical Details of the Experiments

The smooth convex problem and CIFAR-10 airbench Experiments were conducted on a single NVIDIA RTX A4000. The code was run with Python 3.10.13 and Pytorch 2.6.0+cu126 on a computer with an EPYC 7543 processor.



(a) Training accuracy



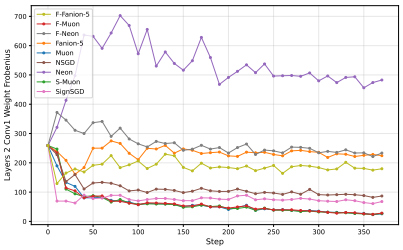
(b) Validation accuracy

Fig. 11 Different optimizers on CIFAR-10 airbench without weight normalization.

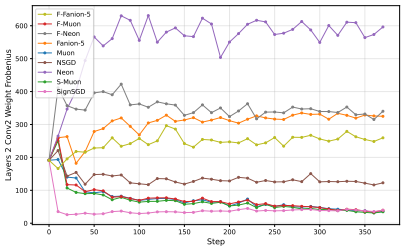
NanoGPT and GPT-2 Medium Pretrain We used the standard setting of 8 × NVIDIA H100 as documented in [18]. The code was run with Python 3.10.18, Pytorch 2.10.0.dev20251124+cu126 on a computer with Xeon® Platinum 8462Y+ processor and 130 GB of RAM.

NanoGPT Fine-tuning. Experiments were conducted on a single NVIDIA RTX 4090 (24GB) GPU using PyTorch 2.8 with CUDA 12.8 and cuDNN 9.0, running on a workstation equipped with an Intel Core i9-14900KS CPU and 128 GB of RAM. No distributed or mixed-hardware training setups were used, ensuring a strictly controlled and unbiased comparison across optimizers.

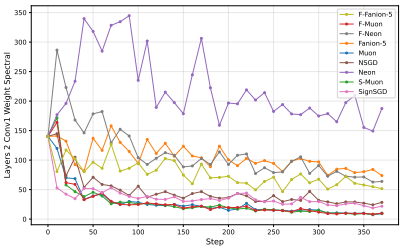
A uniform training protocol was applied to all runs, including an identical batch size, warm-up and cosine decay learning rate schedule, gradient clipping strategy, and validation split. Our optimization methods were integrated into the NanoGPT training loop without modifying the model architecture or preprocessing pipeline. Model quality was primarily evaluated using validation loss tracked over 200 training steps.



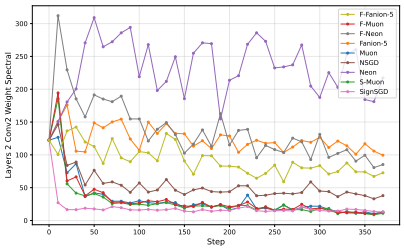
(a) Frobenius norm of layer2.conv1



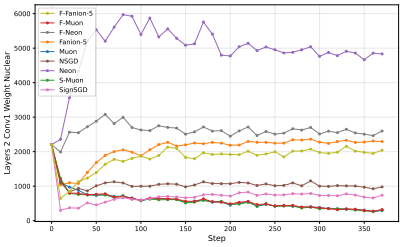
(b) Frobenius norm of layer2.conv2



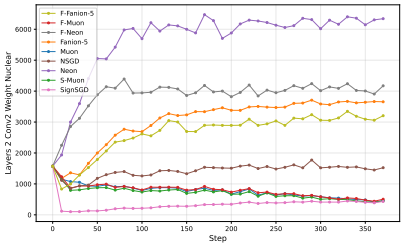
(c) Spectral norm of layer2.conv1



(d) Spectral norm of layer2.conv2

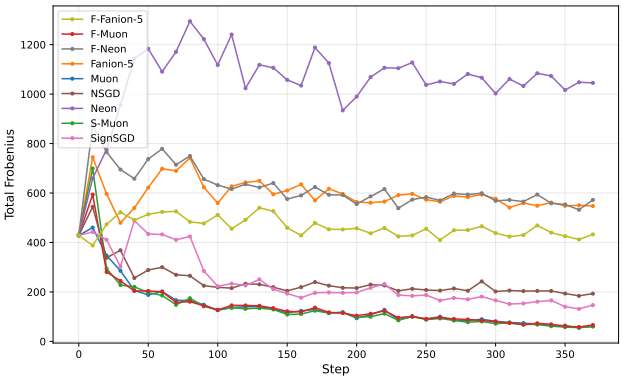


(e) Nuclear norm of layer2.conv1

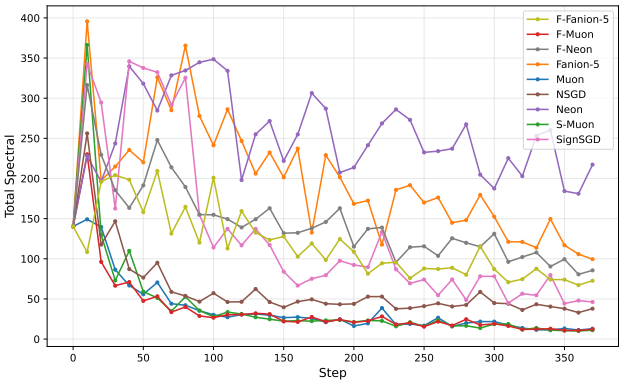


(f) Nuclear norm of layer2.conv2

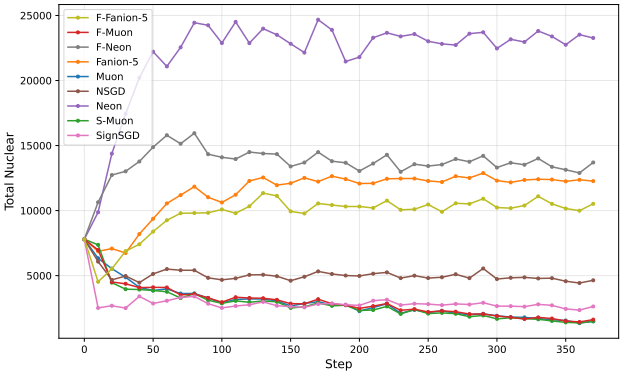
Fig. 12 Matrix norms of layer2.conv1 (left) and layer2.conv2 (right) gradients of CIFAR CNN.



(a) Total Frobenius norm



(b) Total spectral norm



(c) Total nuclear norm

Fig. 13 Matrix norms of the whole gradient of CIFAR CNN.